validating the product each time. If our model is working, the first two should be invalid, and we verify that the error message associated with the `price` attribute is what we expect.

The last price is acceptable, so we assert that the model is now valid. (Some folks would put these three tests into three separate test methods—that's perfectly reasonable.)

Next, we test that we're validating that the image URL ends with one of `gif`, `jpg`, or `png`:

rails7/depot_c/test/models/product_test.rb

```
def new_product(image_url)
  Product.new(title:       "My Book Title",
              description: "yyy",
              price:       1,
              image_url:   image_url)
end


test "image url" do
  ok = %w{ fred.gif fred.jpg fred.png FRED.JPG FRED.Jpg
           http://a.b.c/x/y/z/fred.gif }
  bad = %w{ fred.doc fred.gif/more fred.gif.more }

  ok.each do |image_url|
    assert new_product(image_url).valid?,
           "#{image_url} must be valid"
  end

  bad.each do |image_url|
    assert new_product(image_url).invalid?,
           "#{image_url} must be invalid"
  end
end
```

Here we've mixed things up a bit. Rather than write the nine separate tests, we've used a couple of loops—one to check the cases we expect to pass validation and the second to try cases we expect to fail. At the same time, we factored out the common code between the two loops.